

CARAMEL AI

Full-Stack Chatbot — Complete Learning Program

React

TypeScript

Node.js

Express

Azure OpenAI

Streaming SSE

A beginner-friendly workshop for building your first AI-powered web application

100+ Slides • 15 Sections • Every Line of Code Explained

Workshop Agenda

15 Sections • 100+ Slides • 1 Full Day

01 Project Introduction

02 Frontend Fundamentals

03 TypeScript Deep Dive

04 React Deep Dive

05 App.tsx Line-by-Line

06 JavaScript Deep Dive

07 Backend Fundamentals

08 Express Deep Dive

...and Sections 09-15 →

SECTION 01

Project Introduction

What we are building — and why

What is Caramel AI?

A streaming AI chatbot powered by Azure OpenAI

The Project

Caramel AI is a full-stack web application that lets users chat with an AI assistant in real time. It streams responses word-by-word — just like ChatGPT — using a React frontend and a Node.js + Express backend connected to Azure OpenAI.

What You Will Learn

You will understand every line of code that makes this work — from the React UI to the Express server to the Azure AI API, including streaming responses.

Tech Stack

Frontend: React + TypeScript

Backend: Node.js + Express

AI: Azure OpenAI (GPT)

Protocol: Server-Sent Events (SSE)

What Problem Does It Solve?

Why build a custom AI chatbot?

Data Privacy

Your data stays on your own server, not sent to third-party chat products

Custom Branding

Branded as 'Caramel' with your own persona and personality

Full Control

You decide the system prompt, the model, the rate limits, and the UI

Learning Lab

The best way to truly understand AI APIs is to build on top of one

Learning Objectives

By the end of this workshop, you will be able to

- Explain what React is and build a component from scratch
- Use TypeScript interfaces to define data shapes
- Create a Node.js + Express REST API server
- Make authenticated API calls to Azure OpenAI
- Implement real-time streaming with Server-Sent Events
- Handle async operations with async/await and Promises
- Structure a full-stack project with separated frontend & backend
- Rebuild Caramel AI from scratch without looking at the source code

The Final Product

Here is what you will build today

User Experience

1. User types a message in the input box
2. Presses Enter or clicks Send
3. Message appears on screen immediately
4. AI response streams in, word by word
5. Full conversation history maintained

Under the Hood

- React manages the UI and state
- Fetch API sends message to Express
- Express forwards to Azure OpenAI
- Azure streams tokens back
- Express relays the stream to the browser
- React updates the UI in real time

Key Insight

Every word appears one by one — this is STREAMING. It is not a single API response. It is hundreds of tiny chunks arriving over time.

SECTION 02

Frontend Fundamentals

What runs in the browser — and why React?

What is a Frontend?

The part of the app that users see and interact with

Kid Explanation 🧒

Think of a restaurant. The frontend is the dining room — the tables, the menu, the waiter you talk to. You never see the kitchen.

Technical Explanation 💻

The frontend is code that runs inside the user's web browser (Chrome, Safari, Firefox). It is responsible for: displaying content, handling user interactions, sending data to the server, and updating the screen dynamically.

Frontend Technologies in Our Project

- HTML — structure of the page
- CSS — styling and visual design
- TypeScript/JavaScript — logic and behaviour
- React — component library that manages the UI

Traditional HTML vs React

How did we build UIs before React?

Traditional HTML + JS

- Write HTML manually for every UI state
- Update the DOM by calling `document.getElementById()`
- Code quickly becomes messy (called 'spaghetti')
- Hard to reuse UI components
- State and UI get out of sync easily
- Difficult to maintain as app grows

React

- Describe what the UI should LOOK LIKE
- React figures out what changed and updates the DOM
- Code stays clean and component-based
- Components are reusable building blocks
- State and UI are always in sync automatically
- Scales beautifully to large applications

What is the DOM?

Document Object Model — the browser's model of your page

Kid Explanation 🧒

Imagine your webpage is a family tree. The `<html>` tag is the grandparent. Inside it are children: `<head>` and `<body>`. Inside `<body>` are more children like `<div>`, `<p>`, `<button>`. The DOM is this family tree.

Real DOM vs Virtual DOM

Real DOM: The actual tree in the browser. Updating it is SLOW because the browser repaints pixels.

Virtual DOM: React's in-memory copy of the tree. React computes what changed (called DIFFING), then applies ONLY those changes to the real DOM. This makes React very fast.

Component-Based Architecture

React's fundamental building block

What is a Component?

A component is a self-contained, reusable piece of UI. Think of it like a LEGO brick. You build a complex app by snapping many components together.

```
// A simple React component
function ChatMessage({ text, sender }) {
  return (
    <div className={`message ${sender}`}>
      <p>{text}</p>
    </div>
  );
}

// Use it like an HTML tag!
<ChatMessage text="Hello!" sender="user" />
<ChatMessage text="Hi there!" sender="bot" />
```

JSX

What is a SPA?

Single Page Application — one page that never reloads

Traditional Multi-Page Website

Every time you click a link, the browser makes a new request to the server, downloads a new HTML page, and REPAINTS everything. This causes a full-page flash.

Single Page Application (SPA)

The browser loads ONE HTML page. When you navigate, JavaScript dynamically updates the content without reloading. Feels like a native desktop app. No page flashes.

Caramel AI is a SPA

Our chatbot is a Single Page Application. React renders the chat UI. When you send a message, JavaScript updates the DOM dynamically — no full page reload ever happens.

SECTION 03

TypeScript Deep Dive

Writing safer, smarter JavaScript

What is TypeScript?

JavaScript with superpowers

Kid Explanation 🧒

JavaScript is like writing notes with a pencil — you can write anything. TypeScript is like using a pencil with a spell-checker built in. It warns you when you make a mistake BEFORE the teacher reads it.

Technical Explanation

TypeScript is a SUPERSET of JavaScript, created by Microsoft. It adds optional static type checking. TypeScript code is compiled (transpiled) into regular JavaScript before running in the browser.

Key Benefit

TypeScript catches type errors at COMPILE TIME (when you write code), not at RUNTIME (when users are using your app). This prevents entire categories of bugs.

TypeScript: Type Annotations

Telling TypeScript what type a variable holds

TS

```
// JavaScript (no types – anything goes)
let name = "Ruthran";
name = 42;           // No error in JS! Bug!
name = true;        // Still no error! Another bug!

// TypeScript (types enforced)
let name: string = "Ruthran";
name = 42;          // ❌ Error: Type 'number' not assignable to 'string'

// Common type annotations
let age: number = 25;
let isLoggedIn: boolean = true;
let userName: string = "Caramel";
let tags: string[] = ["ai", "chat"]; // Array of strings
let scores: number[] = [90, 85, 92]; // Array of numbers
```


TypeScript: Interfaces

Defining the shape of an object

What is an Interface?

An interface is like a CONTRACT or a BLUEPRINT. It defines what properties an object must have and what type each property must be.

```
// From App.tsx – Our actual interface!  
interface Message {  
  text: string;           // The message content (must be a string)  
  sender: "user" | "bot"; // Only "user" or "bot" allowed (union type)  
}
```

TS

```
// Now TypeScript enforces this shape everywhere:  
const userMsg: Message = { text: "Hello!", sender: "user" }; //  Valid  
const badMsg: Message = { text: "Hi", sender: "admin" };     //  Error!  
const incomplete: Message = { text: "Hi" };                  //  Error! Missing 'sender'
```

TypeScript: Union Types

A variable that can be ONE of several types

What is a Union Type?

The pipe symbol | means OR. A union type says: this variable can be type A OR type B, but nothing else.

```
// Union type with string literals
type Sender = "user" | "bot";

// This is what happens in our Message interface:
sender: "user" | "bot"

// Valid:
const s1: Sender = "user";    // ✓
const s2: Sender = "bot";    // ✓

// Invalid:
const s3: Sender = "admin";  // ✗ Error!
const s4: Sender = "AI";     // ✗ Error!
const s5: Sender = 42;       // ✗ Error!

// Why this matters:
// Our code uses sender to apply CSS classes
```

TS

TypeScript: Generic Types

Writing reusable, type-safe code

What is a Generic?

A generic is a TYPE PLACEHOLDER. Instead of specifying a fixed type, you use a placeholder (like T) that gets filled in when you use the function or type.

```
// useState from React uses Generics!
// From App.tsx:
const [messages, setMessages] = useState<Message[]>([]);
//
//                               Generic: useState holds an array of Messages

// Without the generic, TypeScript would infer type 'never[]'
// (an empty array with no type)

// With the generic, TypeScript knows:
// - messages is always Message[]
// - setMessages only accepts Message[]
// - Each item must have { text: string, sender: "user"|"bot" }

// Another example – useState<boolean>:
const [loading, setLoading] = useState<boolean>(false);
```

TS

TypeScript: Why It Matters

Catching bugs before they reach your users

Without TypeScript — A Real Bug Scenario

Imagine you call `msg.txt` instead of `msg.text`. In plain JavaScript, this returns undefined and your UI silently shows nothing. You only discover this bug when a user reports it.

```
// With TypeScript — Bug caught immediately at compile time:
const msg: Message = { text: "Hello", sender: "user" };

console.log(msg.txt); // ❌ Error: Property 'txt' doesn't exist on 'Message'
//                  Did you mean 'text'?

console.log(msg.text); // ✅ Works perfectly

// TypeScript = your AI pair programmer that never sleeps
```

TS

SECTION 04

React Deep Dive

The library that powers our frontend UI

What is React?

A JavaScript library for building user interfaces

Kid Explanation 🧒

React is like LEGO. You build small, reusable pieces (components), then snap them together to make something big and complex. Each piece knows how to display itself.

Technical Explanation

React is an open-source JavaScript library created by Facebook (Meta) in 2013. It uses a component-based architecture, a Virtual DOM for performance, and a declarative paradigm: you describe WHAT the UI should look like, React handles HOW to render it.

Core Principles

1. Declarative — Describe the UI state, React renders it
2. Component-Based — Build encapsulated components
3. Learn Once, Write Anywhere — Works for web, mobile (React Native), desktop

What is JSX?

JavaScript + HTML — writing UI in JavaScript

What is JSX?

JSX stands for JavaScript XML. It lets you write HTML-like syntax directly inside JavaScript. React then transforms it into JavaScript function calls.

```
// JSX – What you write:
const element = <h1 className="title">Hello, World!</h1>;

// What React transforms it into (compiled output):
const element = React.createElement("h1", { className: "title" }, "Hello, World!");

// You can embed JavaScript expressions inside JSX using {}:
const name = "Ruthran";
const greeting = <h1>Hello, {name}!</h1>;
// Renders: Hello, Ruthran!

// Conditional in JSX:
const isLoggedIn = true;
const display = <div>{isLoggedIn ? "Welcome back!" : "Please log in"}</div>
```

JSX

React State — useState Hook

The most important React concept

Kid Explanation 🧒

State is React's memory. Imagine a whiteboard that React uses to remember information. When the whiteboard changes, React redraws the screen to reflect the new information.

```
// useState syntax:
const [value, setValue] = useState(initialValue);
//      |           |
//      |           | Starting value
//      |           | Function to update the state
//      |           | Current value of the state

// From App.tsx – Three state variables:
const [messages, setMessages] = useState<Message[]>([]);
//   Current messages
//   Function to update messages

const [input, setInput] = useState("");
//   Current input text

const [loading, setLoading] = useState(false);
```

TSX

React: How State Updates Work

The render cycle explained

The Render Cycle

1. Component renders with initial state
2. User interaction triggers a state change (setMessages)
3. React detects the state change
4. React re-renders the component with the NEW state
5. React diffs the old and new Virtual DOM
6. React updates ONLY the changed parts of the real DOM

Why This Matters for Our Chat

When a new message arrives:
`setMessages(prev => [...prev, newMsg])`

This triggers a re-render.
React sees the new message in the array.
React adds ONLY the new message to the DOM.
The rest of the chat stays untouched.

This is EFFICIENT — React is surgical.

Important Rule

React: Event Handling

Responding to user interactions

What is an Event?

An event is something that happens in the browser: a click, a keypress, a form submission, a mouse hover. React lets you listen for events and run code when they happen.

```
// From App.tsx – Two event handlers:

// 1. onChange – fires every time the input text changes
<input
  value={input}
  onChange={(e) => setInput(e.target.value)}
  // _____
  // e = the event object
  // e.target = the input element
  // e.target.value = what the user typed
/>

// 2. onKeyDown – fires when user presses a key
onKeyDown={(e) => e.key === "Enter" && sendMessage()}
// If the pressed key is Enter, call sendMessage()
```

TSX

React: Controlled Inputs

React owning the input value

What is a Controlled Input?

In a controlled input, React state is the SINGLE SOURCE OF TRUTH for the input's value. The input's value is always what state says it is.

```
// From App.tsx:
const [input, setInput] = useState(""); // State = source of truth

<input
  type="text"
  value={input} // Input always shows what 'input' state says
  onChange={(e) => setInput(e.target.value)} // When user types, update state
  placeholder="Type your message..."
/>

// Flow:
// User types "H" → onChange fires → setInput("H") → input state = "H"
// React re-renders → input.value is now "H" → user sees "H"

// After sending, we reset:
setInput(""); // Clears the input by setting state to empty string
```

TSX

React: Rendering Lists with .map()

Displaying dynamic arrays of data

Why .map()?

Our messages array can have 0, 5, or 100 items. We need to render a UI element for EACH item. The .map() method transforms each item in the array into a React element.

```
// From App.tsx:
{messages.map((msg, index) => (
  <div key={index} className={`message ${msg.sender}`}>
    <span className="sender-tag">
      {msg.sender === "user" ? "You" : "AI"}
    </span>
    <p className="message-text">{msg.text}</p>
  </div>
))}
```

TSX

```
// .map() takes a function. For each 'msg' in 'messages':
// → Creates a <div> with the message content
// → key={index} is REQUIRED — helps React identify which items changed
// → msg.sender === "user" ? "You" : "AI" — ternary to show label
```

SECTION 05

App.tsx — Line by Line

Every line of the React frontend explained

App.tsx — Overview

The complete frontend component

```
import { useState } from 'react';           // Line 1
import './App.css';                         // Line 2

interface Message {                         // Lines 4-7
  text: string;
  sender: "user" | "bot";
}

export default function App() {             // Line 9
  const [messages, setMessages] = useState<Message[]>([]); // L10
  const [input, setInput] = useState("");      // L11
  const [loading, setLoading] = useState(false); // L12

  const sendMessage = async () => { ... };      // L14-68

  return ( <div className="chat-container"> ... </div> ); // L70-87
}
```

TSX

Line 1: import { useState } from 'react'

Importing a hook from the React library

What is an import?

import lets you use code from other files or libraries. React is a library installed via npm. We import just the useState function from it (not the entire library).

```
import { useState } from 'react';  
//  
//      Named import      Package name (installed via npm)  
  
// Why { curly braces }?  
// The React package exports MANY things: useState, useEffect, useRef, etc.  
// Curly braces = "destructured" named import  
// We only import what we need – keeps the bundle smaller  
  
// What is useState?  
// A React Hook – a special function that gives our component "memory"  
// It lets us store and update values that affect the UI
```

TSX

Line 2: import './App.css'

Importing styles for our component

```
import './App.css';  
// This imports the CSS file and applies it globally.  
// There are no named exports here – it's a SIDE EFFECT import.  
// Webpack (the build tool) bundles this CSS and injects it into the page.  
  
// App.css contains styles for our chat UI:  
// .chat-container → the outer wrapper  
// .chat-box      → the scrollable message area  
// .message       → each individual message bubble  
// .message.user  → user messages (right-aligned, blue)  
// .message.bot   → AI messages (left-aligned, gray)  
// .input-area    → the input + button row
```

TSX

Why separate CSS?

Separating styles into .css files keeps concerns separate. JavaScript handles logic. CSS handles appearance. This is called Separation of Concerns — a core software engineering principle.

Lines 4-7: interface Message

Defining the shape of a chat message

```
interface Message {  
  text: string;  
  sender: "user" | "bot";  
}  
  
// Why define this interface?  
// Without it, TypeScript doesn't know what a "message" looks like.  
// With it, every time we create a message object, TypeScript  
// ENFORCES that it has exactly these two properties.  
  
// Union type "user" | "bot":  
// sender can ONLY be the string "user" OR the string "bot"  
// TypeScript will error if you pass any other value  
  
// This interface is used in three places:  
// 1. useState<Message[]>([])      – type the state array  
// 2. const userMessage: Message    – type-check new messages  
// 3. .map((msg, index) =>...)       – TypeScript knows msg.text and msg.sender
```

TSX

Line 9: export default function App()

The main React component

```
export default function App() {  
  // _____  
  // |                Component name (must start with capital letter)  
  // Makes this the default export – other files import it as:  
  //   import App from './App'  
  
  // Why 'function'?  
  // This is a Functional Component (the modern React way).  
  // Old React used Class Components (class App extends React.Component).  
  // Functional components are simpler, shorter, and more powerful  
  // thanks to Hooks.  
  
  // The component must return JSX – HTML-like code that React renders.  
  // Everything inside the return() is what appears on screen.
```

TSX

Naming Convention

Lines 10-12: useState Declarations

Setting up our component's memory

```
const [messages, setMessages] = useState<Message[]>([]);  
// _____  
// messages      → current array of all chat messages (initially empty)  
// setMessages    → function to update the messages array  
// Message[]      → generic: array of Message objects  
// []             → initial value: empty array  
  
const [input, setInput] = useState("");  
// _____  
// input          → current text in the input box (initially "")  
// setInput       → function to update the input text  
// ""             → initial value: empty string  
  
const [loading, setLoading] = useState(false);  
// _____  
// loading        → is the AI currently responding? (initially false)  
// setLoading     → function to set loading to true/false  
// false          → initial value: not loading
```

TSX

Lines 14-15: sendMessage Function

The core async function

```
const sendMessage = async () => {  
  // _____  
  // |           'async' marks this as an asynchronous function  
  // Variable name – stores the function  
  
  // What is 'async'?  
  // Some operations take time: network requests, file reads, timers.  
  // async functions let us WAIT for these operations without freezing the browser.  
  // Inside an async function, we use 'await' to pause and wait for a result.  
  
  // Arrow function syntax:  
  // const sendMessage = async () => { ... }  
  // Is equivalent to:  
  // async function sendMessage() { ... }  
  // Both work. Arrow functions are more commonly used in React.
```

TSX

Lines 16-23: Guard + Message Creation

Preventing empty sends and building messages

```
if (!input.trim() || loading) return;
// _____
// input.trim() → removes whitespace from both ends
// !input.trim() → if the trimmed input is empty string → stop
// || loading → OR if AI is currently responding → stop
// return → exit the function early (guard clause)

const userMessage: Message = { text: input, sender: "user"};
// Creates a new Message object conforming to our interface:
// text: input → the current value of the input state
// sender: "user" → marks this as a user message

const updatedMessages = [...messages, userMessage];
// Spread operator: [...messages] → copy of existing messages
// Then append userMessage to the end
// Result: new array with all old messages PLUS the new one

setMessages(updatedMessages); // Update state → triggers re-render
setInput("");                 // Clear the input box
setLoading(true);              // Show "AI is typing..."
```

TSX

Lines 24-25: Bot Placeholder

Pre-adding the bot's empty message

```
const botPlaceholderIndex = updatedMessages.length;
// _____
// updatedMessages.length = the index where the bot message WILL BE
// If we have 3 messages, the next message goes at index 3
// We save this index so we can UPDATE this slot later

setMessages((prev) => [...prev, { text: "", sender: "bot" }]);
// _____
// This adds an empty bot message to the chat RIGHT NOW.
// Why? So the UI shows "AI is typing..." immediately.
// As streaming tokens arrive, we REPLACE this empty message
// with the growing bot reply.

// This is the STREAMING UI TRICK:
// 1. Add empty bot message slot
// 2. As tokens stream in, fill it up progressively
// User sees text appearing word by word – feels like live typing!
```

TSX

Lines 27-31: Fetch API Call

Sending the request to our backend

```
const response = await fetch("http://localhost:5001/chat", {  
  //  
  //      'await' pauses here until fetch completes  
  //      _____  
  //      URL of our Express server endpoint  
  
  method: "POST",  
  //  
  //      HTTP Method: POST sends data in the body (not in the URL)  
  
  headers: { "Content-Type": "application/json"},  
  //  
  //      Tells the server we're sending JSON data  
  
  body: JSON.stringify({ messages: updatedMessages })  
  //  
  //      Convert JS object to JSON string: { "messages": [...] }  
  //      fetch() sends this as the request body  
});
```

TSX

Lines 32-38: Reading the Stream

Consuming Server-Sent Events

```
if (!response.ok) { throw new Error("Network response was not ok"); }  
// response.ok is true if HTTP status is 200-299. Throw if error.  
  
const reader = response.body?.getReader();  
// response.body → ReadableStream (the stream of data)  
// ?. → optional chaining: only call .getReader() if body exists  
// .getReader() → get a reader to consume the stream chunk by chunk  
  
const decoder = new TextDecoder("utf-8");  
// TextDecoder converts raw bytes (Uint8Array) into text strings  
// "utf-8" is the encoding – the standard for web text  
  
if (!reader) throw new Error("No reader available");  
  
let botReply = "";  
// Accumulates the bot's response as tokens stream in  
// Starts empty, grows with each streaming chunk
```

TSX

Lines 40-68: The Streaming Loop

Reading tokens one by one as they arrive

```
while (true) {                                     // Loop forever until 'done'
  const { done, value } = await reader.read();
  //      _____
  //      |         |                               Wait for next chunk of data
  //      |         |                               Raw bytes (Uint8Array)
  //      |         |                               true when stream is finished
  if (done) break;                                // Exit loop when stream ends

  const chunkStr = decoder.decode(value);           // Bytes → string

  const lines = chunkStr.split("\n");              // Split by newline

  for (const line of lines) {                      // Process each line
    if (!trimmed.startsWith("data: ")) continue; // SSE format
    const dataJson = trimmed.replace("data: ", "");
    if (dataJson === "[DONE]") continue;          // End signal

    const parsed = JSON.parse(dataJson);
    const content = parsed.choices?.[0]?.delta?.content || "";
    if (content) {
      botReply += content;                          // Accumulate text
      setMessages((prev) => {                        // Update UI
        const next = [...prev];
        next[hotPlaceholderIndex] = { text: botReply, sender: "bot" };
      });
    }
  }
}
```

TSX

SSE Format: What's in Each Chunk?

Understanding the data: lines

Server-Sent Events (SSE) Format

The Azure OpenAI streaming API uses SSE format. Each chunk is a line starting with 'data: ' followed by JSON, or the special '[DONE]' signal.

```
// A raw SSE chunk might look like:
data: {"choices":[{"delta":{"content":"Hello"},"index":0}]}

data: {"choices":[{"delta":{"content":" there"},"index":0}]}

data: {"choices":[{"delta":{"content":""},"index":0}]}

data: [DONE]

// Our parsing code:
trimmed.startsWith("data: ") // Check it's a data line
.replace("data: ", "")       // Strip the "data: " prefix
JSON.parse(dataJson)         // Parse the JSON
parsed.choices?.[0]?.delta?.content // Extract the text token
// Optional chaining ?. → safe even if any field is missing
```

JSON

The JSX Return Block

What actually renders on screen

```
return (

TSX



<div className="chat-container">
      <h2>Caramel AI</h2>
      <div className="chat-box">
        {messages.map((msg, index) => (
          <div key={index} className={`message ${msg.sender}`}>
            <span className="sender-tag">
              {msg.sender === "user" ? "You" : "AI"}
            </span>
            <p className="message-text">{msg.text}</p>
          </div>
        ))}
        {loading && <div className="message bot loading">AI is typing...</div>}
      </div>
      <div className="input-area">
        <input type="text" value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Type your message..."
          onKeyDown={(e) => e.key === "Enter" && sendMessage()} />
        <button onClick={sendMessage} disabled={loading}>Send</button>
      </div>
    </div>
  )


```

SECTION 06

JavaScript Deep Dive

The core language concepts powering our app

const vs let vs var

Declaring variables in modern JavaScript

```
// const – Cannot be reassigned. Used for values that don't change.
const PORT = 5001;           // Always 5001
const SYSTEM_PROMPT = "..."; // Always the same prompt

// let – Can be reassigned. Used for values that change.
let botReply = "";           // Starts empty, grows with each token
let buffer = '';             // Accumulates incoming stream data

// var – OLD. Don't use it.
// var has 'function scope' and 'hoisting' – causes subtle bugs.
// const and let have 'block scope' – more predictable.

// Rule of thumb:
// Use const by default.
// Only use let if you KNOW you'll reassign the variable.
// Never use var in modern code.
```

JS

Arrow Functions

The modern way to write functions in JavaScript

JS

```
// Traditional function declaration:  
function sendMessage() { ... }
```

```
// Arrow function expression:  
const sendMessage = () => { ... }
```

```
// With parameters:  
const add = (a, b) => a + b;           // One-line: implicit return  
const greet = (name) => {               // Multi-line: explicit return  
  const msg = "Hello, " + name;  
  return msg;  
};
```

```
// From App.tsx:  
const sendMessage = async () => { ... } // No params  
onChange={e => setInput(e.target.value)} // One param, inline  
(msg, index) => ( <div key={index}> )    // Two params, JSX
```

```
// Key difference: 'this' binding  
// Arrow functions don't have their own 'this' – they inherit it.  
// This matters in class components (rare in modern React).
```

Async/Await & Promises

Handling time-consuming operations

Kid Explanation 🧒

Imagine ordering a pizza. You don't stand at the door waiting (blocking). You go watch TV and the delivery person calls when it arrives (async).
async/await is JavaScript's way to 'wait for the pizza'.

```
// The old way – Promise chains (hard to read):
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// The modern way – async/await (reads like synchronous code):
async function getData() {
  try {
    const response = await fetch(url); // Wait for fetch
    const data = await response.json(); // Wait for JSON parse
    console.log(data);
  } catch (err) {
    console.error(err); // Handle any error
  }
}
```

JS

Spread Operator

Copying and combining arrays/objects

JS

```
// The spread operator: ...
// Expands an array or object into individual elements

// ARRAYS:
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]
// Creates a NEW array – original unchanged!

// From App.tsx:
const updatedMessages = [...messages, userMessage];
//
//           Copy all existing messages
//           Add the new one

// Also used in state updates:
const next = [...prev]; // Copy the array
next[botPlaceholderIndex] = newMessage; // Modify the copy
return next; // Return the modified copy

// OBJECTS:
const obj1 = { name: "Ruthran", age: 25 };
const obj2 = { ...obj1, city: "Chennai" };
// { name: "Ruthran", age: 25, city: "Chennai" }
```


Optional Chaining (?.) and Nullish Coalescing (||)

Safe property access and defaults

JS

```
// Optional Chaining: ?.  
// Access nested properties safely – returns undefined if anything is null  
  
// Without optional chaining (dangerous):  
const content = parsed.choices[0].delta.content;  
// If choices is undefined → TypeError: Cannot read property '0' of undefined  
  
// With optional chaining (safe):  
const content = parsed.choices?.[0]?.delta?.content || "";  
//           ↑           ↑           ↑  
//           Each ?. returns undefined if left side is null/undefined  
  
// Nullish coalescing: ||  
// Provides a default value if the left side is falsy  
const content = parsed.choices?.[0]?.delta?.content || "";  
//           ───  
//           If content is undefined/null/"", use ""  
  
// From server.js:  
const reader = response.body?.getReader();  
// Only call getReader() if response.body is not null
```

Template Literals

String interpolation in JavaScript

JS

```
// Old way – string concatenation:
const greeting = "Hello, " + name + "! You have " + count + " messages.";
// Messy, especially with many variables

// New way – template literals (backticks):
const greeting = `Hello, ${name}! You have ${count} messages.`;
//           ↑           ↑
//           Backtick (`)   Expression in ${...}

// From App.tsx:
<div className={`message ${msg.sender}`}>
// Combines "message " with either "user" or "bot"
// Result: "message user" or "message bot"

// From server.js:
const url = `${endpoint}/openai/deployments/${deployment}/chat/completions?api-version=${apiVersion}`;
// Builds the Azure OpenAI URL dynamically from env variables

// Template literals also support multi-line strings:
const multiLine = `Line 1
Line 2
Line 3`;
```

Array Destructuring

Unpacking array values into variables

```
// Destructuring: unpack values from arrays/objects into variables
```

JS

```
// Array Destructuring:
```

```
const [a, b] = [1, 2];
```

```
// a = 1, b = 2
```

```
// This is exactly how useState works:
```

```
const [messages, setMessages] = useState<Message[]>([]);
```

```
//
```

```
// First element: current value
```

```
// Second element: updater function
```

```
// The useState function returns an array of TWO things:
```

```
// [currentValue, setterFunction]
```

```
// We destructure it to give them readable names
```

```
// Another example from the streaming loop:
```

```
const { done, value } = await reader.read();
```

```
// Object destructuring (with curly braces)
```

```
// Extracts 'done' and 'value' properties from the returned object
```

JSON — JavaScript Object Notation

The language of web APIs

```
// JSON is the standard format for sending data between frontend and backend.
```

JS

```
// JavaScript Object:
```

```
const message = { text: "Hello", sender: "user" };
```

```
// Convert to JSON string (for sending over network):
```

```
const jsonString = JSON.stringify(message);
```

```
// '{"text":"Hello","sender":"user"}' ← string, not object
```

```
// Parse JSON string back to object:
```

```
const parsed = JSON.parse(jsonString);
```

```
// { text: "Hello", sender: "user" } ← back to object
```

```
// In our app:
```

```
// FRONTEND → sends JSON to backend:
```

```
body: JSON.stringify({ messages: updatedMessages })
```

```
// BACKEND → receives JSON:
```

```
app.use(express.json()) // Parses incoming JSON automatically
```

```
// BACKEND → receives JSON from Azure:
```

```
const parsed = JSON.parse(dataJson) // Parse each SSE chunk
```

SECTION 07

Backend Fundamentals

What runs on the server — Node.js and beyond

What is Node.js?

JavaScript that runs on the server

Kid Explanation 🧒

Usually, JavaScript only runs in your browser (Chrome, Safari). Node.js is like a magic box that lets JavaScript run on a computer server — the machine that hosts your website.

Technical Explanation

Node.js is a JavaScript runtime built on Chrome's V8 engine. Released in 2009, it allows JavaScript to run outside the browser. It uses an event-driven, non-blocking I/O model, making it ideal for building APIs and real-time applications.

Why Node.js for Our Backend?

- Same language (JS/TS) on frontend and backend
- Excellent for real-time streaming (our key feature)
- Huge ecosystem (npm) with packages for everything
- Fast for I/O-bound tasks like API calls
- Industry-standard for chat applications

The Event Loop

How Node.js handles multiple tasks at once

Kid Explanation 🧒

Imagine a chef in a restaurant. Instead of cooking one dish completely before starting the next, they put dish A in the oven, start dish B, check dish A, stir dish C. They never just WAIT — they keep working.

Technical Explanation

Node.js is single-threaded. The Event Loop is what makes it handle many requests without blocking. When Node.js makes an I/O call (network, file), it registers a callback and moves on. When the I/O completes, the callback is put in the event queue and executed.

Why This Matters

When our Express server calls Azure OpenAI, it doesn't freeze and wait. It registers a callback, handles OTHER incoming requests, and processes the Azure response when it arrives. This is why Node.js can handle many concurrent users with low resources.

Client-Server Architecture

How frontend and backend communicate



The Request-Response Cycle

1. User types a message in React (Browser)
2. React calls `fetch()` to send an HTTP POST request
3. Express receives the request and processes it
4. Express forwards the request to Azure OpenAI
5. Azure streams tokens back to Express
6. Express streams tokens to the Browser
7. React updates the UI in real time

SECTION 08

Express Deep Dive

Building REST APIs with Node.js and Express

What is Express?

The most popular Node.js web framework

Kid Explanation 🧒

Express is like a traffic controller for your server. When requests come in from the browser, Express decides which code to run based on the URL path and HTTP method.

Technical Explanation

Express.js is a minimal, flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It handles routing, middleware, request/response management, and HTTP utilities.

```
// Minimal Express server:
import express from 'express';
const app = express();           // Create an Express application
app.use(express.json());         // Parse JSON request bodies

// Define a route:
app.get('/hello', (req, res) => {
  res.json({ message: 'Hello!' });
});

app.listen(3000, () => console.log('Running on port 3000'));
```

JS

Express Middleware

Functions that run before your route handler

What is Middleware?

Middleware is a function that sits between the incoming request and your route handler. It can read/modify the request, perform authentication, log data, parse JSON, etc.

```
// Middleware signature:
app.use((req, res, next) => {
  // req = request object
  // res = response object
  // next = function to call the NEXT middleware
  console.log(`${req.method} ${req.path}`);
  next(); // Pass control to the next middleware
});

// From server.js – two middleware:
app.use(cors()); // CORS middleware: allows browser requests
// Without this, browser blocks cross-origin requests!

app.use(express.json()); // JSON middleware: parses req.body
// Without this, req.body would be undefined
```

JS

What is CORS?

Cross-Origin Resource Sharing explained

The Problem

Browsers have a security rule: JavaScript on `http://localhost:3000` (React) cannot make requests to `http://localhost:5001` (Express) because they have DIFFERENT ORIGINS (different ports). The browser blocks it for security.

The Solution

The server tells the browser: 'I allow requests from your origin'. The `cors()` middleware adds these permission headers to every response. The browser sees the header and allows the request.

```
// Without CORS:  
// Browser blocks request:  
// "Access to fetch at 'http://localhost:5001'  
// has been blocked by CORS policy"
```

JS

```
// With CORS middleware:  
import cors from 'cors';  
app.use(cors()); // ✅ Browser now allows  
requests
```

```
// The header added:
```

⚠️ Production Note

`app.use(cors())` allows ALL origins. In production, specify: `app.use(cors({ origin: 'https://yourapp.com' }));` to only allow your own frontend.

HTTP Methods & Status Codes

The vocabulary of web APIs

HTTP Methods

- GET — Retrieve data (no body)
- POST — Send data to server
- PUT — Replace an entire resource
- PATCH — Partial update of a resource
- DELETE — Remove a resource
- Our app only uses POST /chat

Status Codes

- 200 OK — Request succeeded
- 201 Created — Resource created
- 400 Bad Request — Invalid input
- 401 Unauthorized — Not authenticated
- 403 Forbidden — No permission
- 404 Not Found — Resource missing
- 500 Internal Server Error — Server crash

SECTION 09

server.js — Line by Line

Every line of the Node.js backend explained

server.js — Overview

The complete backend server file

```
import express from 'express';           // Line 1
import cors from 'cors';                 // Line 2
import dotenv from 'dotenv';              // Line 3

dotenv.config();                          // Line 5

const app = express();                   // Line 7
app.use(cors());                         // Line 8
app.use(express.json());                  // Line 9

const PORT = 5001;                       // Line 11

const SYSTEM_PROMPT = "You are Caramel, assistant to Ruthran" // L13

app.post('/chat', async (req, res) => { // L15 – The main route
  // Validate request
  // Read environment variables
  // Build Azure OpenAI URL
  // Forward request to Azure
  // Stream response back to client
});

app.listen(PORT, () => { ... });          // L69 – Start the server
```

JS

Lines 1-3: ES Module Imports

Modern JavaScript import syntax

```
import express from 'express';  
// 'express' is a package installed via: npm install express  
// This imports the Express constructor function  
  
import cors from 'cors';  
// 'cors' package: npm install cors  
// Adds CORS headers so our React frontend can make requests  
  
import dotenv from 'dotenv';  
// 'dotenv' package: npm install dotenv  
// Reads environment variables from a .env file  
  
// Note: Using ES module syntax (import/export)  
// This requires "type": "module" in package.json  
// Alternative: CommonJS syntax (const express = require('express'))  
  
// Why install packages? Node.js core is minimal by design.  
// npm (Node Package Manager) gives you 2 million+ packages to extend it.
```

JS

Line 5: dotenv.config()

Loading environment variables

```
dotenv.config();

// What does this do?
// Reads the .env file from your project root:
//   .env file contains:
//   AZURE_FOUNDRY_ENDPOINT=https://your-resource.openai.azure.com
//   AZURE_FOUNDRY_KEY=your-secret-api-key-here
//   AZURE_FOUNDRY_DEPLOYMENT=gpt-4o

// After dotenv.config(), these become available via:
process.env.AZURE_FOUNDRY_ENDPOINT // The Azure endpoint URL
process.env.AZURE_FOUNDRY_KEY      // The secret API key
process.env.AZURE_FOUNDRY_DEPLOYMENT // The model deployment name

// Why .env files?
// ✔ Secrets stay out of your source code
// ✔ .env is added to .gitignore – never committed to GitHub
// ✔ Different values in development vs. production
// ✗ NEVER hardcode API keys in your code!
```

JS

Lines 7-13: App Setup & Constants

Express app, middleware, and configuration

```
const app = express();  
// Creates an Express application instance.  
// 'app' is an object with methods: .use(), .get(), .post(), .listen()  
  
app.use(cors());  
// Adds CORS middleware globally – runs before every route handler  
// Allows our React frontend (different port) to make requests  
  
app.use(express.json());  
// Parses JSON request bodies automatically  
// Without this: req.body would be undefined  
// With this: req.body = { messages: [...] }  
  
const PORT = 5001;  
// Port number the server will listen on.  
// 5001 (not 3000) because Create React App runs on 3000 by default.  
  
const SYSTEM_PROMPT = "You are Caramel, assistant to Ruthran";  
// System prompt = AI's personality/instructions  
// This tells the AI who it is and how to behave  
// Sent with every API call as the first message with role: "system"
```

JS

Lines 15-22: The POST /chat Route

Handling incoming chat requests

```
app.post('/chat', async (req, res) => {  
  //  
  //      URL path      Route handler function  
  //      async: because we'll await Azure API call  
  
  const { messages } = req.body;  
  //  
  //      Destructure The parsed JSON body  
  //      Gets the messages array the frontend sent  
  
  if (!messages || !Array.isArray(messages)) {  
    return res.status(400).json({ error: 'Missing message' });  
    //  
    //      400 = Bad Request  
    //      Return early with error if messages is missing/invalid  
  }  
}
```

JS

Why validate?

Lines 23-30: Reading Env Variables

Configuring the Azure connection

```
const endpoint    = process.env.AZURE_FOUNDRY_ENDPOINT;  
const key        = process.env.AZURE_FOUNDRY_KEY;  
const deployment = process.env.AZURE_FOUNDRY_DEPLOYMENT;  
const apiVersion = '2024-05-01-preview';  
  
// Build the Azure OpenAI API URL:  
const url = `${endpoint}/openai/deployments/${deployment}/chat/completions?api-version=${apiVersion}`;  
  
// Example URL built:  
// https://myresource.openai.azure.com/openai/deployments/gpt-4o/chat/completions?api-version=2024-05-01-  
// preview  
// _____  
// Endpoint from .env           Deployment from .env           Fixed API version  
  
// Why hardcode apiVersion?  
// The API version controls which Azure OpenAI features are available.  
// We use a specific known-working version for stability.
```

JS

Lines 31-44: Transforming Messages

Converting our format to Azure's format

JS

```
// Our messages format (from React):
// [{ text: "Hello", sender: "user" }, { text: "Hi!", sender: "bot" }]

// Azure OpenAI requires this format:
// [{ role: "user", content: "Hello" }, { role: "assistant", content: "Hi!" }]

const azureMessages = messages.map((msg) => ({
  role: msg.sender === 'user' ? 'user' : 'assistant',
  // _____
  //   Convert 'sender: "bot"' → 'role: "assistant"'
  //   Azure uses "assistant" not "bot"
  content: msg.text,
  //   Convert 'text: "..."' → 'content: "..."'
})));

// Add system prompt at the beginning:
azureMessages.unshift({ role: 'system', content: SYSTEM_PROMPT });
// _____
//   unshift() inserts at INDEX 0 (beginning of array)
//   System prompt must be FIRST in the messages array
```

Lines 45-57: Calling Azure OpenAI

The API request that powers the AI

```
const response = await fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'api-key': key, // Azure uses 'api-key' header (not Bearer token)
  },
  body: JSON.stringify({
    messages: azureMessages, // The transformed conversation
    stream: true, // ← KEY: Enable streaming!
  }),
});

if (!response.ok) {
  const errorText = await response.text();
  throw new Error(`Azure OpenAI Error: ${response.status} - ${errorText}`);
}

// The 'stream: true' parameter tells Azure to:
// ✅ Send response tokens immediately as they're generated
// ✅ Use SSE (Server-Sent Events) format
// Instead of waiting for the full response, we get words one by one
```

JS

Lines 58-66: Setting SSE Headers

Preparing the streaming response

```
res.writeHead(200, {  
  // _____  
  // Write HTTP response headers immediately (before sending body)  
  'Content-Type': 'text/event-stream',  
  // _____  
  // SSE content type: tells browser to expect a stream  
  
  'Cache-Control': 'no-cache',  
  // _____  
  // Don't cache – stream data is real-time, never stale  
  
  'Connection': 'keep-alive',  
  // _____  
  // Keep the HTTP connection open (don't close after one response)  
  // Required for SSE – connection stays open while tokens stream  
});  
  
// Without these headers, the browser would:  
// - Close the connection after the first response  
// - Try to cache the streaming data  
// - Not interpret it as a live stream
```

JS

Lines 67-85: The Server Streaming Loop

Relaying Azure tokens to the browser

```
const reader = response.body.getReader(); // Read Azure's stream
const decoder = new TextDecoder('utf-8'); // Bytes → text
let buffer = ''; // Line buffer

while (true) {
  const { done, value } = await reader.read(); // Get next chunk
  if (done) break;

  buffer += decoder.decode(value, { stream: true });
  //
  // Important: stream:true handles
  // multi-byte characters across chunks

  const lines = buffer.split('\n');
  buffer = lines.pop() || ''; // Keep incomplete last line in buffer
  //
  // A chunk might end mid-line. We save it for next iteration.

  for (const line of lines) {
    if (line.trim() === '') continue;
    res.write(`${line}\n\n`); // Forward line to browser in SSE format
  }
}
```

JS

Lines 86-90: Finishing the Stream

Flushing the buffer and closing the connection

JS

```
if (buffer) res.write(`${buffer}\n\n`);  
// _____  
//           After the loop, there might be leftover data in buffer.  
//           Send it to the client before closing.  
  
res.end();  
// _____  
// Close the HTTP connection.  
// This signals to the browser: "the stream is done"  
// The frontend's reader.read() will then return { done: true }  
  
// app.listen():  
app.listen(PORT, () => {  
  console.log(`Server is running on port ${PORT}`);  
});  
// Starts the HTTP server.  
// The callback runs once the server is ready to accept connections.  
// ${PORT} = 5001 – browsers connect to http://localhost:5001
```

SECTION 10

Azure OpenAI

The AI brain powering Caramel

What is Azure OpenAI?

OpenAI's models, hosted on Microsoft Azure

OpenAI

The company that created GPT-4, ChatGPT, and DALL-E. OpenAI provides these models via an API.

Azure

Microsoft's cloud computing platform. Hosts servers, databases, AI services, and more for businesses worldwide.

Azure OpenAI

OpenAI's models (GPT-4, GPT-4o) hosted on Azure's infrastructure. Adds enterprise features: compliance, SLAs, private networking.

Why Azure OpenAI instead of OpenAI directly?

- Enterprise security and compliance requirements
- Data doesn't leave your Azure region (data residency)
- Integration with other Azure services
- Custom enterprise SLAs and support
- Often required by corporate IT policies

Key Azure OpenAI Concepts

Endpoint, Deployment, API Key, API Version

Endpoint

The base URL of your Azure resource. Format: `https://YOUR-RESOURCE.openai.azure.com`. Each Azure account has a unique endpoint.

Deployment

A named instance of a model. You create a deployment called 'gpt-4o' and point it at the GPT-4o model. Your code references the deployment name, not the model directly.

API Key

A secret password that proves you're authorized to use the Azure resource. Never share this. Keep it in `.env`.

API Version

The version of the Azure OpenAI REST API. New versions add features but may break old code. We pin to '2024-05-01-preview'.

The Chat Completion API

How we send and receive AI messages

JSON

```
// The Chat Completions endpoint takes an array of messages:
{
  "messages": [
    { "role": "system", "content": "You are Caramel, assistant to Ruthran" },
    { "role": "user", "content": "What is React?" },
    { "role": "assistant", "content": "React is a JavaScript library..." },
    { "role": "user", "content": "Can you give me an example?" }
  ],
  "stream": true
}

// Three message roles:
// system    → Instructions to the AI (comes first, sets personality)
// user      → What the human said
// assistant → What the AI previously said

// Full conversation history is sent with EVERY request.
// Azure OpenAI doesn't remember previous messages by itself!
// We must send the full context each time.
```

System Prompts & Prompt Engineering

Shaping the AI's personality and behaviour

What is a System Prompt?

The system prompt is a special instruction message with role: 'system'. It sets the AI's persona, capabilities, restrictions, and tone. The AI follows it throughout the conversation.

```
// Our system prompt (from server.js):  
const SYSTEM_PROMPT = "You are Caramel, assistant to Ruthran";  
  
// More powerful example:  
const SYSTEM_PROMPT = `You are Caramel, a friendly AI assistant.  
Rules:  
- Always be concise and helpful  
- Speak in a warm, professional tone  
- If you don't know something, say so honestly  
- Focus on technical questions about programming  
- Address the user as Ruthran`;  
  
// Prompt Engineering = crafting prompts to get better AI responses.  
// The quality and specificity of your system prompt dramatically  
// affects the quality of the AI's responses.
```

JS

SECTION 11

Streaming Architecture

How tokens flow from Azure to your screen

The Complete Streaming Flow

End-to-end data journey

1

User Sends Message

React calls `fetch()` with POST to `/chat`, sending the full message history as JSON

2

Express Receives

Express parses the JSON body, validates it, transforms format, adds system prompt

3

Azure Streams

Express calls Azure OpenAI with `stream:true`. Azure starts sending tokens immediately

4

Express Relays

Express reads Azure's `ReadableStream` and writes each SSE line to the browser response

5

React Updates UI

Why Streaming? The UX Impact

Why not just wait for the full response?

Without Streaming

- User sends message
- Browser shows loading spinner
- User waits 5-15 seconds
- Full response appears all at once
- Feels slow and unresponsive
- User doesn't know if anything is happening
- Poor user experience

With Streaming

- User sends message
- First words appear in < 1 second
- Response builds word by word
- User reads as AI 'thinks'
- Feels fast and alive
- User knows AI is working
- ChatGPT-like experience

Parsing SSE: The Detail

How our code reads streaming tokens

```
// A raw response chunk from Azure might contain multiple lines:
"data: {\\"choices\\":[{\\"delta\\":{\\"content\\":\\"Hello\\"},...}]]\\n\\n"
"data: {\\"choices\\":[{\\"delta\\":{\\"content\\":\\" there\\"},...}]]\\n\\n"
"data: [DONE]\\n\\n"

// Our parsing pipeline:
const chunkStr = decoder.decode(value);           // Bytes → string
const lines = chunkStr.split("\\n");              // Split into lines
for (const line of lines) {
  const trimmed = line.trim();
  if (!trimmed.startsWith("data: ")) continue; // Skip non-data lines
  const dataJson = trimmed.replace("data: ", ""); // Strip prefix
  if (dataJson === "[DONE]") continue;          // Skip done signal
  const parsed = JSON.parse(dataJson);          // Parse JSON
  const content = parsed.choices?.[0]?.delta?.content || "";
  // Optional chaining: safe if any field is missing
  if (content) {
    botReply += content; // Append to accumulated reply
    // Update UI...
  }
}
```

JS

SECTION 12

Project Architecture

How all the pieces fit together

High-Level Architecture

The three layers of Caramel AI

Presentation Layer

React + TypeScript
App.tsx • CSS

API Layer

Express + Node.js
server.js

AI Layer

Azure OpenAI
GPT Model

Frontend

Vite + React 18, TypeScript, CSS Modules, fetch API,
ReadableStream

Backend

Node.js, Express.js, dotenv, cors, ES Modules, SSE

AI Integration

Azure OpenAI Service, GPT-4o, Chat Completions API,
Streaming

Dev Tools

npm, .env, .gitignore, TypeScript compiler, Vite dev server

File Structure

How the project is organized on disk

```
caramel-ai/
├── frontend/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── App.css
│   │   └── main.tsx
│   ├── index.html
│   ├── package.json
│   └── tsconfig.json
├── backend/
│   ├── server.js
│   ├── .env
│   ├── .gitignore
│   └── package.json
└── README.md
```

← React app (created with Vite)

← Main component (what we studied)

← Chat styles

← React entry point

← HTML shell

← Frontend dependencies

← TypeScript config

← Node.js server

← Express server (what we studied)

← 🔒 Secret keys (NEVER commit!)

← Excludes .env, node_modules

← Backend dependencies

← Project documentation

bash

SECTION 13

Security Best Practices

Keeping secrets secret

Why API Keys Must Stay on the Backend

The most critical security rule

✗ WRONG: Key in Frontend

- API key visible in browser DevTools
- Key exposed in GitHub if committed
- Anyone can steal and use your key
- You pay for unauthorized API usage
- Your Azure account could be compromised
- Potentially expensive security breach

undled in
use it to c

✓ CORRECT: Key on Backend

- API key stays on the server
- Browser never sees the key
- Users communicate with YOUR server
- Your server communicates with Azure
- Completely hidden from end users
- Industry-standard secure architecture

Environment Variables & .env Files

Managing secrets safely

```
// .env file (in your backend/ folder):
AZURE_FOUNDRY_ENDPOINT=https://myresource.openai.azure.com
AZURE_FOUNDRY_KEY=sk-abc123-this-is-your-secret-key
AZURE_FOUNDRY_DEPLOYMENT=gpt-4o

// .gitignore file – CRITICAL:
node_modules/
.env           ← Tells git to NEVER track this file
.env.local
.env.*.local

// Accessing in code:
process.env.AZURE_FOUNDRY_KEY // "sk-abc123-..."

// For production deployment:
// Set environment variables in your hosting platform
// (Azure App Service, AWS, Heroku, Vercel, etc.)
// Never use .env files in production!

// For team members (share template, not values):
// Create .env.example with placeholder values:
AZURE_FOUNDRY_ENDPOINT=https://YOUR-RESOURCE.openai.azure.com
AZURE_FOUNDRY_KEY=YOUR-API-KEY-HERE
```

bash

Common Security Mistakes

What NOT to do — and how to fix it

Hardcoded API Keys

✗ `const key = 'sk-abc123...';`

✓ Use `process.env.API_KEY` from `.env`

Committing `.env` to Git

✗ `git add .env && git commit`

✓ Add `.env` to `.gitignore` FIRST

Storing key in React code

✗ `fetch(url, { headers: { 'api-key': KEY } })`

✓ Call your own Express backend instead

Using `cors()` with no restrictions

✗ `app.use(cors())` in production

✓ Specify exact origin: `cors({ origin: 'https://yourapp.com' })`

SECTION 14

Troubleshooting Guide

When things go wrong — and how to fix them

Common React & TypeScript Errors

Error messages decoded

Cannot read properties of undefined

Accessing a property on undefined. Check optional chaining: `obj?.property`

Property 'X' does not exist on type 'Y'

TypeScript type error. Check your interface definition or update the type.

React Hook rules violation

Hooks must be called at the top level. Not inside if statements or loops.

Maximum update depth exceeded

Infinite render loop. State update inside `useEffect` without dependency array.

JSX expressions must have one parent element

Wrap multiple elements in a `<div>` or React Fragment (`<>...</>`).

Common Node.js & Express Errors

Backend debugging guide

Cannot find module 'express'

npm install express — package not installed

CORS error: blocked by CORS policy

Add `app.use(cors())` before your routes in `server.js`

SyntaxError: Cannot use import outside module

Add `'type': 'module'` to `package.json`

process.env.KEY is undefined

`dotenv.config()` not called, or `.env` file missing/misnamed

Port 5001 already in use

Kill the existing process: `lsof -ti:5001 | xargs kill`

Common Azure OpenAI Errors

AI API error guide

401 Unauthorized

api-key header is wrong or missing. Check .env and verify the key.

404 Not Found

Endpoint URL is wrong, or deployment name doesn't match. Check all three env vars.

429 Too Many Requests

Rate limit exceeded. Add delays or request a quota increase in Azure portal.

Stream cuts off mid-response

Network timeout. Check keep-alive settings and increase timeout limits.

Content filtered or empty response

Azure Content Filter blocked the content. Review your prompt for policy violations.

SECTION 15

Future Improvements

Where to take Caramel AI next

Authentication & User Management

Who is using your chatbot?

Current State

Currently, anyone with the URL can use Caramel AI with no login.
This means:

- No user-specific chat history
- No rate limiting per user
- Costs could run high from abuse

With Authentication

- Auth0 or Firebase Auth for login
- JWT tokens to verify user identity
- Per-user conversation history
- Rate limiting by user
- Usage analytics
- Subscription management

Technologies to Add

Auth0 (OAuth), Firebase Auth, NextAuth.js (if moving to Next.js), Clerk.dev, Supabase Auth. All provide drop-in authentication for React apps.

Persistent Chat History & Database

Remembering conversations across sessions

Current Limitation

When you refresh the page, all chat history is LOST. React state only lives in memory — when the browser reloads, it starts fresh.

Relational Database

PostgreSQL or MySQL to store:

- Users table (id, email, name)
- Conversations table (id, user_id, title)
- Messages table (id, conversation_id, role, content, timestamp)

Document Database

MongoDB stores conversations as JSON documents — more flexible for chat logs. Use Mongoose (ODM) for Node.js integration. MongoDB Atlas for cloud hosting.

RAG: Retrieval-Augmented Generation

Teaching your AI about your own data

The Problem with Standard AI

GPT-4 knows about the world up to its training cutoff. It knows NOTHING about your company's internal documents, your product specs, your personal notes, or any private data.

What is RAG?

RAG lets you:

1. Upload your documents (PDFs, text, Word files)
2. Convert them to vector embeddings (numerical representations)
3. Store in a vector database
4. When user asks a question, find relevant document chunks
5. Send those chunks as context to the AI with the question
6. AI answers based on YOUR documents!

Technologies

Vector DBs: Pinecone, Weaviate, ChromaDB, pgvector

Embeddings: Azure OpenAI ada-002

Orchestration: LangChain, LlamaIndex

This is the foundation of enterprise AI applications.

AI Agents & Multi-Agent Systems

AI that can take actions, not just answer questions

AI Agents

Instead of just generating text, AI Agents can use TOOLS to take real actions:

- Search the web in real time
- Write and execute code
- Send emails
- Query databases
- Call APIs
- Browse websites

Multi-Agent Systems

Multiple AI agents collaborate to solve complex tasks:

- Orchestrator agent breaks down the problem
- Specialist agents handle subtasks
- Agents share results and build on each other
- More powerful than any single agent

Other Advanced Features to Explore

Voice Chat (Azure Speech Services) • File Uploads & Analysis • Image Generation • Markdown Rendering for code/equations • Multi-modal AI (text + images) • Fine-tuning models on your data

What You Learned Today

The complete picture

React & TypeScript

Components, hooks, state, JSX, interfaces, generics

JavaScript

Async/await, spread, arrow functions, destructuring, fetch

Node.js & Express

Server setup, middleware, routing, JSON APIs, SSE

Azure OpenAI

Endpoints, deployments, chat completions, streaming

Full Stack

How frontend and backend communicate over HTTP

Security

Environment variables, API key protection, CORS

Your Homework: Build It From Scratch

The true test of understanding

Challenge

Close the source code files. Open a blank editor. Build Caramel AI from scratch using only your notes and this presentation. No copy-pasting.

1. `npm create vite@latest frontend -- --template react-ts`
2. Create `src/App.tsx` with `useState` for messages, input, loading
3. Build the `sendMessage` async function with `fetch` + streaming
4. Create `backend/server.js` with `Express` + `cors` + `dotenv`
5. Set up the `POST /chat` route with Azure OpenAI call
6. Add SSE headers and stream the response back
7. Create `.env` with your Azure credentials
8. Test: `npm run dev` (frontend) + `node server.js` (backend)

You Did It!

You now understand how Caramel AI works — every line of code.

React • TypeScript • Node.js • Express • Azure OpenAI • Streaming

Keep building. Keep learning. Keep shipping.

— Caramel AI Workshop